

02-02 정제(Cleaning) and 정규화(Normalization)



코퍼스에서 용도에 맞게 토큰을 분류하는 작업을 토큰화(tokenization)라고 하며, 토큰화 작업 전, 후에는 텍스트 데이터를 용도에 맞게 정제(cleaning) 및 정규화(normalization)하는 일이 항상 함께합니다. 정제 및 정규화의 목적은 각각 다음과 같습니다.

- 정제(cleaning): 갖고 있는 코퍼스로부터 노이즈 데이터를 제거한다.
- 정규화(normalization): 표현 방법이 다른 단어들을 통합시켜서 같은 단어로 만들어준다.

정제 작업은 토크와 작업에 방해가 되는 부분들을 배제시키고 토크와 작업을 수행하기 위해서 토크와 작업보다 앞서 이루어지기도 하지만, 토크와 작업 이후에도 여전히 남아있는 노이즈들을 제거하기 위해 지속적으로 이루어지기도 합니다. 사실 완벽한 정제 작업은 어려운 편이라서, 대부분의 경우 이 정도면 됐다.라는 일종의 합의점을 찾기도 합니다.

1. 규칙에 기반한 표기가 다른 단어들의 통합

필요에 따라 직접 코딩을 통해 정의할 수 있는 정규화 규칙의 예로서 같은 의미를 갖고 있음에도, 표기가 다른 단어들을 하나의 단어로 정규화하는 방법을 사용할 수 있습니다.

가령, USA와 US는 같은 의미를 가지므로 하나의 단어로 정규화해 볼 수 있습니다. uh-huh와 uhhuh는 형태는 다르지만 여전히 같은 의미를 갖고 있습니다. 이러한 정규화를 거치게 되면, US를 찾아도 USA도 함께 찾을 수 있을 것입니다. 뒤에서 표기가 다른 단어들을 통합하는 방법인 어간 추출(stemming)과 표제어 추출(lemmatization)에 대해서 더 자세히 알아봅니다.

2. 대, 소문자 통합

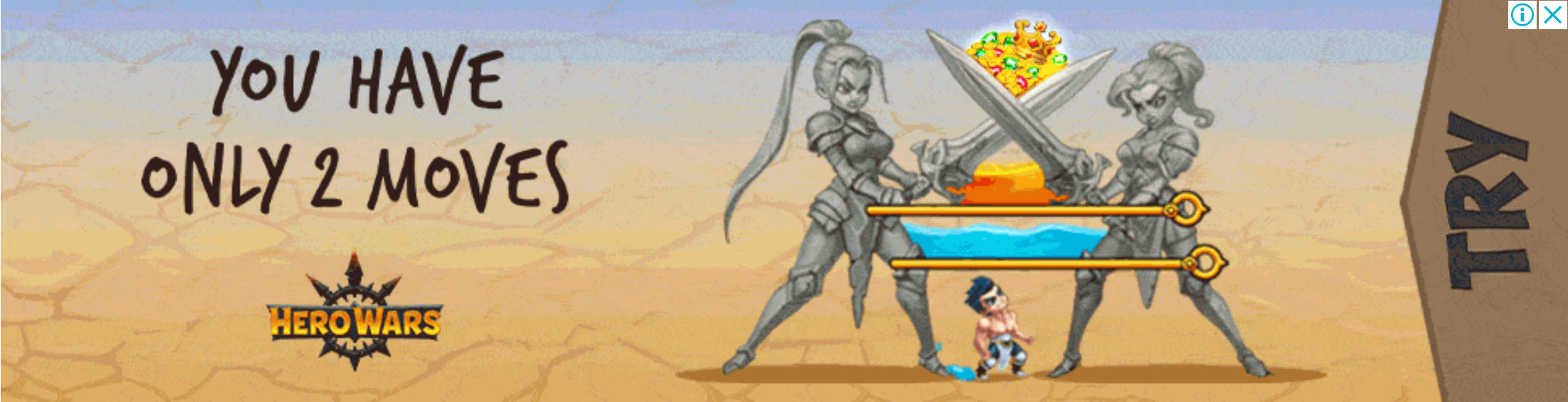
영어권 언어에서 대, 소문자를 통합하는 것은 단어의 개수를 줄일 수 있는 또 다른 정규화 방법입니다. 영어권 언어에서 대문자는 문장의 맨 앞 등과 같은 특정 상황에서만 쓰이고, 대부분의 글은 소문자로 작성되기 때문에 대, 소문자 통합 작업은 대부분 대문자를 소문자로 변환하는 소문자 변환작업으로 이루어지게 됩니다.

소문자 변환이 왜 유용한지 예를 들어보도록 하겠습니다. 가령, Automobile이라는 단어가 문장의 첫 단어였기 때문에 A가 대문자였다고 생각해보십시오. 여기에 소문자 변환을 사용하면, automobile을 찾는 질의(query)의 결과로서 Automobile도 찾을 수 있게 됩니다. 검색 엔진에서 사용자가 페라리 차량에 관심이 있어서 페라리를 검색해본다고 합시다. 엄밀히 말해서 사실 사용자가 검색을 통해 찾고자하는 결과는 a Ferrari car라고 봐야합니다. 하지만 검색 엔진은 소문자 변환을 적용했을 것이기 때문에 ferrari만 쳐도 원하는 결과를 얻을 수 있을 것입니다.

물론 대문자와 소문자를 무작정 통합해서는 안 됩니다. 대문자와 소문자가 구분되어야 하는 경우도 있습니다. 가령 미국을 뜻하는 단어 US와 우리를 뜻하는 us는 구분되어야 합니다. 또 회사 이름(General Motors)나, 사람 이름(Bush) 등은 대문자로 유지되는 것이 좋습니다.

모든 토큰을 소문자로 만드는 것이 문제를 가져온다면, 또 다른 대안은 일부만 소문자로 변환시키는 방법도 있습니다. 가령, 이런 규칙은 어떨까요? 문장의 맨 앞에서 나오는 단어의 대문자만 소문자로 바꾸고, 다른 단어들은 전부 대문자인 상태로 놔두는 것입니다.

이러한 작업은 더 많은 변수를 사용해서 소문자 변환을 언제 사용할지 결정하는 머신 러닝 시퀀스 모델로 더 정확하게 진행시킬 수 있습니다. 하지만 만약 올바른 대문자 단어를 얻고 싶은 상황에서 훈련에 사용하는 코퍼스가 사용자들이 단어의 대문자, 소문자의 올바른 사용 방법과 상관없이 소문자를 사용하는 사람들로부터 나온 데이터라면 이러한 방법 또한 그다지 도움이 되지 않을 수 있습니다. 결국에는 예외 사항을 크게 고려하지 않고, 모든 코퍼스를 소문자로 바꾸는 것이 종종 더 실용적인 해결책이 되기도 합니다.



3. 불필요한 단어의 제거

정제 작업에서 제거해야하는 노이즈 데이터(noise data)는 자연어가 아니면서 아무 의미도 갖지 않는 글자들(특수 문자 등)을 의미하기도 하지만, 분석하고자 하는 목적에 맞지 않는 불필요 단어들을 노이즈 데이터라고 하기도 합니다.

불필요 단어들을 제거하는 방법으로는 불용어 제거와 등장 빈도가 적은 단어, 길이가 짧은 단어들을 제거하는 방법이 있습니다. 불용어 제거는 불용어 첵트에서 더욱 자세히 다루기로 하고, 여기서는 등장 빈도가 적은 단어와 길이가 짧은 단어들을 제거하는 경우에 대해서 간략히 설명하겠습니다.

(1) 등장 빈도가 적은 단어

때로는 텍스트 데이터에서 너무 적게 등장해서 자연어 처리에 도움이 되지 않는 단어들이 존재합니다. 예를 들어 입력된 메일이 정상 메일인지 스팸 메일인지를 분류하는 스팸 메일 분류기를 설계한다고 가정해보겠습니다. 총 100,000개의 메일을 가지고 정상 메일에서는 어떤 단어들이 주로 등장하고, 스팸 메일에서는 어떤 단어들이 주로 등장하는지를 가지고 설계하고자 합니다. 그런데 이 때 100,000개의 메일 데이터에서 총 합 5번 밖에 등장하지 않은 단어가 있다면 이 단어는 직관적으로 분류에 거의 도움이 되지 않을 것임을 알 수 있습니다.

(2) 길이가 짧은 단어

영어권 언어에서는 길이가 짧은 단어를 삭제하는 것만으로도 어느정도 자연어 처리에서 크게 의미가 없는 단어들을 제거하는 효과를 볼 수 있다고 알려져 있습니다. 즉, 영어권 언어에서 길이가 짧은 단어들은 대부분 불용어에 해당됩니다. 사실 길이가 짧은 단어를 제거하는 2차 이유는 길이를 조건으로 텍스트를 삭제하면서 단어가 아닌 구두점들까지도 한꺼번에 제거하기 위함도 있습니다. 하지만 한국어에서는 길이가 짧은 단어라고 삭제하는 이런 방법이 크게 유효하지 않을 수 있는데 그 이유에 대해서 정리해보도록 하겠습니다.

단정적으로 말할 수는 없지만, 영어 단어의 평균 길이는 6~7 정도이며, 한국어 단어의 평균 길이는 2~3 정도로 추정되고 있습니다. 두 나라의 단어 평균 길이가 몇 인지에 대해서는 확실히 말하기 어렵지만 그럼에도 확실한 사실은 영어 단어의 길이가 한국어 단어의 길이보다는 평균적으로 길다는 점입니다.

이는 영어 단어와 한국어 단어에서 각 한 글자가 가진 의미의 크기가 다르다는 점에서 기인합니다. 한국어 단어는 한자어가 많고, 한 글자만으로도 이미 의미를 가진 경우가 많습니다. 예를 들어 '학교'라는 한국어 단어를 생각해보면, 배울 학(學)과 학교 교(校)로 글자 하나, 하나가 이미 함축적인 의미를 갖고있어 두 글자만으로 학교라는 단어를 표현합니다. 하지만 영어의 경우에는 학교라는 글자를 표현하기 위해서는 c, h, o, o, i라는 총 6개의 글자가 필요합니다. 다른 예로는 전설 속 동물인 용(龍)을 표현하기 위해서는 한국어로는 한 글자면 충분하지만, 영어에서는 d, r, a, g, o, n이라는 총 6개의 글자가 필요합니다.

이러한 특성으로 인해 영어는 길이가 2~3 이하인 단어를 제거하는 것만으로도 크게 의미를 갖지 못하는 단어를 줄이는 효과를 갖고 있습니다. 예를 들어 갖고 있는 텍스트 데이터에서 길이가 1인 단어를 제거하는 코드를 수행하면 대부분의 자연어 처리에서 의미를 갖지 못하는 단어인 관사 'a'와 주어로 쓰이는 'I'가 제거됩니다. 마찬가지로 길이가 2인 단어를 제거한다고 하면 it, at, to, on, in, by 등과 같은 대부분 불용어에 해당되는 단어들이 제거됩니다. 필요에 따라서는 길이가 3인 단어도 제거할 수 있지만, 이 경우 fox, dog, car 등 길이가 3인 명사들이 제거 되기 시작하므로 사용하고 하는 데이터에서 해당 방법을 사용해도 되는지에 대한 고민이 필요합니다.

```
import re
text = "I was wondering if anyone out there could enlighten me on this car."
```

```
# 길이가 1~2인 단어들을 정규 표현식을 이용하여 삭제
shortword = re.compile(r'\W*\b\w{1,2}\b')
print(shortword.sub('', text))
```

was wondering anyone out there could enlighten this car.

4. 정규 표현식(Regular Expression)



언어엔 코퍼스에서 노이즈 데이터의 특징을 잡아낼 수 있다면, 정규 표현식을 통해서 이를 제거할 수 있는 경우가 많습니다. 가령, HTML 문서로부터 가져온 코퍼스라면 문서 여기저기에 HTML 태그가 있습니다. 뉴스 기사를 크롤링 했다면, 기사마다 게재 시간이 적혀져 있을 수 있습니다. 정규 표현식은 이러한 코퍼스 내에 계속해서 등장하는 글자들을 규칙에 기반하여 한 번에 제거하는 방식으로 매우 유용합니다.

이 책에서도 전처리를 위해 정규 표현식을 앞으로 종종 사용하게 될 겁니다. 예를 들어 위에서 길이가 짧은 단어를 제거할 때도, 정규 표현식이 유용하게 사용되었습니다. 정규 표현식에 대한 자세한 내용은 뒤에서 좀 더 상세하게 다룹니다.